

Radar-based Sentiment Analysis and Recognition

Mohamed-anas Abbouchi¹, Haider Azam², Mudassar Iqbal³

¹Aalto University

²Universitat Politècnica de Catalunya

³Technische Universität Braunschweig

Abstract

Sentiment analysis enables systems to infer human emotional states and adapt their responses. While existing sentiment analysis methods predominantly rely on text, speech, or visual data, body movement remains a comparatively under-explored modality despite its strong connection to human affective expression. Radar sensing offers a promising alternative, as it is non-intrusive, privacy-preserving, and capable of capturing subtle body movements and gestures without the limitations associated with cameras or wearable devices. Using multimodal RF-Behavior dataset, this paper investigates the feasibility of radar-based sentiment recognition through three complementary methodologies. The first approach extracts kinematic, behavioral, and spatial features from radar signals to train classical machine learning classifiers. The second employs a transformer-based architecture trained on radar and infrared sensor data to learn discriminative representations directly from the raw measurements. The third introduces a neural network framework that jointly performs radar signal interpolation and sentiment classification. By comparing these approaches, this work provides a comprehensive evaluation of radar sensing as a modality for sentiment analysis and demonstrates its potential for advancing privacy-preserving affective computing.

1 Introduction

Sentiment analysis commonly refers to the process of identifying and extracting subjective information from text, speech, or other forms of data [Ain *et al.*, 2017]. In this study, we focused on sentiment analysis using radar-based data. Radar sensors are a powerful tool for capturing subtle movements and gestures. They are non-intrusive, privacy-preserving, and much easier to deploy in real-world scenarios compared to other modalities such as cameras or wearable sensors [Sun *et al.*, 2020]. The data from these sensors have proven useful in applications such as gesture recognition [Sun *et al.*, 2020], object detection [Manjunath *et al.*, 2018] and human activity recognition [Froehlich *et al.*, 2024].

The main focus of researchers in the field of sentiment analysis has been on text-based and speech-based data, with a significant amount of work done on NLP models [Rokade *et al.*, 2025] and speech processing techniques. The objective was always to create models that understand human emotions through media but rarely through their body movements. Consequently, the use of radar-based data for sentiment analysis and recognition is still a relatively unexplored area, with limited research and few datasets available. The characteristics of radar signals, such as their ability to capture micro-movements and gestures, make them a promising source of information for micro-movement based experiments such as sentiment analysis [Sun *et al.*, 2020].

The objective of this paper is to explore the potential of radar-based data for sentiment analysis and recognition. We propose different approaches to transform radar signals into training data and evaluate how effective the classification models are in recognizing different sentiments. We split the paper into three different sections, each focusing on a different aspect of the problem and using a different methodology. The first focuses on feature-based classification using a mix of kinematic, behavioral and spatial features extracted from the radar signals, the second method is based on deep learning with raw sensor data, and the third relies on neural network-based interpolation and classification.

2 Related Work

This research is based on the experiments conducted by [Zuo *et al.*, 2025], in which they introduced "a multimodal radio frequency dataset" for both human behavior recognition and emotion analysis. By conducting a series of experiments on 44 participants, they collected data from 13 radars, 8 RFID tags, LoRa devices and infrared cameras in 4 different campaigns that each focused on a different aspect of human behavior. We will be interested in this paper on identifying and classifying the sentiments in the third campaign that was built around 6 sentiment states: *depression, distraction, excitement, focus, relaxation and stress*.

With the nature of the data collected, multiple approaches can be used to analyze it. The initial approach to handle radar data is to extract features from the radar signals and use them as input to classification models. This approach has been used in previous studies for human activity recognition [Deng *et al.*, 2018] and gesture recognition [Sun *et al.*, 2020].

The features extracted from the radar signals can capture the kinematic, behavioral and spatial characteristics of the target’s movement, which can be useful for sentiment analysis. Another approach is the application of deep learning. Recent self-supervised learning methods have shown that powerful representations can be learned directly from raw data without manual feature engineering. SimCLR [Chen *et al.*, 2020] learns embeddings by maximizing agreement between augmented views of the same sample, producing representations that transfer well to downstream classification tasks. CLIP [Radford *et al.*, 2021] extends this idea by aligning embeddings from different modalities into a shared semantic space using contrastive learning. In the language domain, transformer architectures [Vaswani *et al.*, 2017] have demonstrated the effectiveness of self-attention for modeling complex relationships within sequential data. Motivated by these advances, our method applies transformer-based encoders directly to raw radar and infrared camera data, allowing spatial and temporal patterns to be learned automatically rather than relying on handcrafted features. The final approach is to use deep learning models for interpolation which can then be used for classification and forecasting [?], interpolation is very helpful as it can fill in missing data which is common in sensors [?]. Interpolation also helps in converting irregular time-series to a time-series with regularly spaced intervals.

3 Dataset

As previously mentioned, the focus point of this paper is the third campaign of the RF-Behavior dataset and more specifically the radar data. It is composed of a sequence of XYZ coordinates representing the position of a target in 3D space over time. Each method will use a different approach to transform the data into a format suitable for training classification models. However, before any transformation can be applied, the data must first be synchronized across different radar sensors to ensure that the data from different sensors is aligned in time, as this will allow for accurate feature extraction. Once the data is synchronized, the data is normalized on a user basis so the user-specific biases are removed. After these preprocessing steps, the data is ready to be transformed into a format suitable for training classification models.

Even when the radar data is the main concern, the infrared data can be used to provide additional information about the target’s movement and behavior. The infrared data is composed of a sequence of 3 or 6 different 7D landmarks (3 position coordinates and 4 orientation quaternion) which track the target’s body parts over time at a sampling rate of 100 Hz. This data can localize the bodyparts with high precision and will mainly be utilized in the second method that is based on deep learning with raw sensor data.

4 Method 1: Feature-Based Classification

4.1 Preprocessing

For the preprocessing step of this method, we can proceed to segment the data into smaller windows. This is done to capture the temporal dynamics of the target’s movement. The window size and stride are hyperparameters that can be tuned based on the specific application and dataset. In this study, we

Table 1: Extracted trajectory features.

Feature	Description
Path Efficiency	Displacement/trajectory length
Mean Speed	Average speed
Std. Speed	Speed variability
Median Speed	Median speed
25th/75th Speed	Speed quartiles
Mean Acceleration	Average acceleration
Std. Acceleration	Acceleration variability
Median Acceleration	Median acceleration
25th/75th Acceleration	Acceleration quartiles
Mean Jerk	Average jerk
Std. Jerk	Jerk variability
Median Jerk	Median jerk
25th/75th Jerk	Jerk quartiles
Spectral Centroid	Spectrum center of mass
Spectral Entropy	Spectrum complexity
Speed CV	Normalized speed variability
Direction Change Rate	Rate of direction changes
Speed Autocorrelation	Temporal speed correlation
Tortuosity	Path curvature
Speed Skewness	Speed distribution asymmetry
Fraction Stationary Time	Stationary time proportion
Mean Turn Angle	Average turning angle
Turn Angle Entropy	Turning angle diversity
Explained Variance Ratio 1–3	PCA variance ratios

experimented with different window sizes and strides to find the optimal configuration for our classification task. Lower window sizes tend to underperform and higher window sizes tend to overfit the data and lose generalization. The optimal window size was found to be 90 frames with a stride of 20 frames. This configuration allows for capturing the temporal dynamics of the target’s movement while also providing enough data for training the classification models.

4.2 Feature Extraction

Given the preprocessed data, we can extract a set of features that capture the kinematic, behavioral and spatial characteristics of the target’s movement. The features are extracted from each window of data and are used as input to the classification models. The features are divided into three categories: motion dynamics, statistical and directional features. The motion dynamics features capture the speed, acceleration and jerk of the target’s movement. The statistical features capture the distribution of the target’s movement in terms of speed, acceleration and jerk. The directional features capture the directionality of the target’s movement in terms of turn angles and explained variance ratios. The extracted features are summarized in Table 1.

4.3 Training Loop

With the current setup, the data is now composed of 27 features per radar sensor. Given that we have 13 radar sensors, the total number of features is 351 over more than 600 windows. For an effective training of the classification models, we need to reduce the dimensionality of the data and select the most relevant features. To find the optimal configuration of window size, stride and number of features, we implemented a training loop that iterates over different configurations and evaluates the performance of the classification models using Leave-One-Subject-Out (LOSO) cross-validation. It uses the ANOVA F-test to select the top-*k* features for

Algorithm 1 Feature Selection and LOSO Classification

```
for each window configuration  $(w, s)$  do
  for each feature count  $k \in K$  do
    for each LOSO split do
      Split data into training and testing sets.
      for each radar do
        Select top- $k$  features (ANOVA F-test).
      end for
      Concatenate selected features.
      for each classifier  $m \in \mathcal{M}$  do
        Train, evaluate, and record accuracy and
        weighted F1-score.
      end for
    end for
    Compute mean metrics across LOSO folds.
  end for
end for
Select the best  $(w, s, k)$  for each classifier.
Output: Optimal configuration and corresponding accu-
racy and weighted F1-score.
```

each radar sensor and concatenates them to form the final feature set. ANOVA F-test is a statistical method used to determine the significance of differences between group means [Perangin-Angin and Bachtiar, 2021], it is widely used in feature selection tasks where the goal is to identify the most informative features for classification. The classification models are then trained on the training set and evaluated on the test set. The accuracy and weighted F1-score are recorded for each configuration and the best configuration is selected based on the highest mean accuracy across all folds. The training loop is summarized in Algorithm 1.

4.4 Model selection

The classification models considered in this study include Support Vector Machine (SVM), Gradient Boosting (LightGBM, XGBoost, CatBoost) and Multi-Layer Perceptron (MLP). SVM is a popular classification model that finds the optimal hyperplane that separates the data into different classes [Kazemi *et al.*, 2022]. Gradient Boosting is an ensemble learning method that combines multiple weak classifiers to create a strong classifier [Ryan and Massaron, 2025]. MLP is a type of neural network that consists of multiple layers of interconnected nodes [Alsmadi *et al.*, 2009]. The hyperparameters of each model are tuned using grid search and the best configuration is selected based on the highest mean accuracy across all folds.

4.5 Results

Figure 1 compares the performance of the evaluated classifiers using the mean F1-score across all folds. Among the considered models, the Support Vector Machine (SVM) achieved the highest performance with a mean F1-score of 0.72 which suggests that it was able to identify robust decision boundaries.

The gradient boosting models also achieved competitive results. Both LightGBM and XGBoost obtained mean F1-

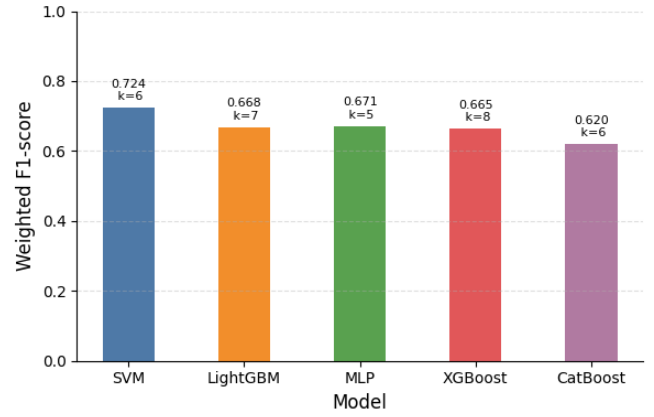


Figure 1: Models comparison based on F1 score.

scores between 0.66 and 0.67, indicating that ensemble tree-based methods are capable of capturing the non-linear relationships present in the extracted features. Their performance, while slightly lower than that of the SVM, confirm that boosting-based approaches can be effective for radar-based sentiment recognition tasks. However, CatBoost underperformed compared to the other gradient boosting models, achieving a mean F1-score of 0.62. While it is known for its strong performance on heterogeneous tabular datasets [Ryan and Massaron, 2025], the specific characteristics of the radar-based features may have limited its effectiveness in this context.

The Multi-Layer Perceptron (MLP) achieved a similar performance to the gradient boosting models, with a mean F1-score in the same range. Although neural networks are generally effective at learning complex feature interactions, the relatively limited size of the dataset and the use of engineered features may have limited the advantages typically associated with deep learning approaches.

Overall, the superior performance of the SVM highlights the effectiveness of the proposed feature extraction pipeline and suggests that carefully engineered motion descriptors can provide sufficient discriminative power without requiring more complex deep learning architectures.

5 Method 2: Deep Learning with Raw Data

5.1 Dataset

For this method, we utilize the radar and infrared camera from the first three campaigns of the RF-Behavior dataset. Behaviors with highly repetitive actions (e.g., walking and running from campaign 2, and the six emotions from campaign 3) are segmented into random intervals of 4 to 6 seconds to match the typical duration of other activities. This segmentation introduces diversity and prevents the model from memorizing irrelevant long-term patterns, such as a participant's specific walking path, rather than learning the actual behavior itself.

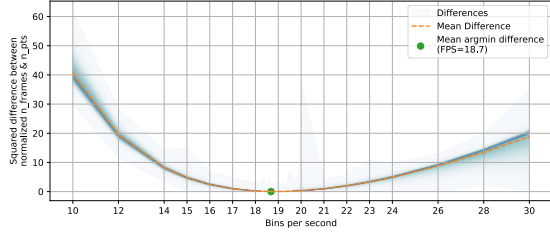


Figure 2: Optimization curve showing how we balance the number of radar frames and spatial points per frame.

5.2 Preprocessing: Uniform Batches

Sampling Rate

Each of the 13 radar sensors captures an irregular number of 4D points (comprising density and X, Y, Z coordinates) at arbitrary timestamps. To structure this sparse data into uniform batches, represented as (batch, time, points, coordinates), we group the data into evenly spaced time bins. We calculate the optimal frame rate empirically by testing values from 10 to 30 frames per second (FPS) at 0.1 increments across all recordings. Across all candidate FPS, we normalize the total number of frames and the average number of points per frame, and compute the squared difference between the two corresponding quantities. This optimization yields a U-shaped error curve (Figure 2) with a minimum at 18.7 FPS (representing a 53.5 ms time bin). At this rate, the average recording’s frame count matches its average points-per-frame count, balancing the temporal and spatial dimensions of our input tensor.

Jagged Input Tensors

Because recording durations vary across samples, the resulting tensors are uneven (jagged) over time. We resolve this by padding shorter sequences on the right using copies of the recording’s final frame. In this work, we did not explore the application of other padding strategies.

Another challenge is that the number of spatial points per frame varies based on active sensors and the specific campaign. Only three upper-body landmarks are tracked via the infrared camera in campaign 1, whereas campaigns 2 and 3 add three lower-body landmarks. Furthermore, individual radar sensors often return varying amounts of data depending on the participant’s distance. To handle this structural variability without locking the network into a fixed layout, we divide each batch of recordings into smaller mini-batches of individual frames that share an identical point count. These uniform subsets are processed independently through the network before being reassembled and combined into the final output tensors.

5.3 Model

Baseline Architecture: Transformers

We design a neural network architecture using two sequential Transformer stacks to condense the variable time and spatial point dimensions (see Figure 3) to size one.

To process the points dimension, we build a Point Cloud Encoder (PCE) that uses a linear projection layer followed by

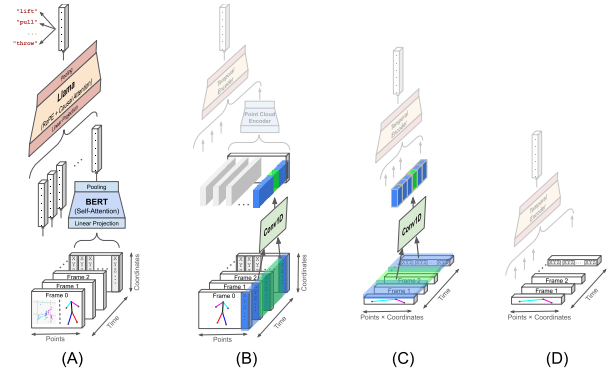


Figure 3: Deep learning pipelines. The model uses a spatial encoder to process 3D points in each frame and a temporal encoder to process sequence of latent vectors. All configurations work with the infrared camera, but only configuration (A) is compatible with radar.

a BERT [Devlin *et al.*, 2019] self-attention encoder. The PCE treats the points in a single frame as a sequence of tokens, learning their spatial relationships to produce a single vector for that frame. Because spatial points have no sequential order, we omit positional encodings for radar. For the infrared camera, we can use learned positional embeddings added to the landmark projections to help the model identify specific body parts. The PCE maps inputs of shape (batch, points, coordinates) to a representation of shape (batch, hidden_size).

To process the time dimension, we implement a Temporal Encoder consisting of a linear projection layer followed by a LLaMA [Touvron *et al.*, 2023] decoder. To capture how physical gestures unfold over time, this encoder uses rotary positional embeddings and causal attention to model sequential relationships across frames. We apply a pooling operation over the output vectors to generate a single summary embedding for the entire recording. An attention mask ensures that the padding frames added during preprocessing are ignored. The temporal encoder processes inputs of shape (batch, time, hidden_size) and outputs a tensor of shape (batch, hidden_size).

For the infrared camera, we also design an optional 1D convolutional downsampling block with a stride of 2 to halve the temporal frame rate and improve efficiency. This block can first flatten all coordinates of all landmarks per frame into a single vector (Configuration 3C), but that requires a fixed sensor setup. Alternatively, Configuration 3B processes each landmark independently using shared weights, allowing the use of PCE to handle any number of tracking points.

Architectural Hyperparameters

To optimize this highly scalable Transformer architecture, we define a search space for its key hyper-parameters (Table 2). We calculate model sizes for 10,000 randomly sampled configurations and group them into 20 geometrically spaced bins ranging from 70k to 100M parameters. We then sequentially process one random model from each bin and evaluate its performance.

Hyperparameter	Values
# Convolutional Blocks	0-4
# Convolutional Channels	2^i ($i = 4, \dots, 7$)
# Transformer blocks	1-6
# Attention Heads	2^i ($i = 2, \dots, 5$)
Embedding Size	2^i ($i = 4, \dots, 10$)
Pooling Strategy	Mean, CLS, EOS
Positional Embeddings (infrared BERT)	Yes/No

Table 2: Architectural hyper-parameters evaluated during grid search.

Supervised Classification

We evaluate and tune these configurations using a supervised classification task. To avoid leakage of an individual’s quirks, we partition the dataset by user IDs. The test set contains four random users, the validation set contains two, and the remaining users are allocated to the training set (approximately 160 recordings per behavior class). We set output projection size of the above model to the number of behavior classes. The network is trained using cross-entropy loss, the AdamW optimizer (learning rate of 10^{-3} , batch size of 32), and early stopping with a patience of 5 epochs. We save the model with the lowest validation loss and evaluate it on the test set using a weighted F1-score. All experiments were run on an NVIDIA GeForce RTX 4070 Laptop GPU (8 GB GDDR6 VRAM).

Unsupervised Contrastive Learning

To make the model useful beyond the specific behaviors in the RF-Behavior dataset, we evaluate its ability to learn general-purpose features without relying on rigid class labels. We map the final output to a 256-dimensional space and train the network in a semi-supervised manner using the NT-Xent loss from SimCLR [Chen *et al.*, 2020] with the rest of the training parameters held the same as the supervised approach. This loss function pulls the embedding vectors of different recordings of the same behavior closer together in vector space, while pushing different behaviors far apart. We evaluate this approach using 1-shot classification on two unseen test behaviors, holding out another two for validation. The NT-Xent loss for a positive pair (i, j) and temperature τ is defined as:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} 1_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}, \quad (1)$$

5.4 Results

Our hyperparameter search revealed that using classification token (CLS) pooling in the PCE collapses the model’s output space, mapping all inputs to nearly identical vectors. Conversely, using convolutional blocks to downsample the temporal dimension in the infrared camera model drastically reduces training time, as the self-attention mechanism’s computational cost scales quadratically with sequence length. For radar models, a larger PCE increases inference time due to our mini-batched-frames by mini-batched-frames processing of jagged points dimension. For unsupervised models, mean pooling consistently out-performed end-of-sequence (EOS) pooling. We also observed that, for unsupervised

Campaign	# Classes	Modality	# Params	Test F1	# Models
C1	21	Radar	1,311,125	94.41%	12
C1	21	Infrared	526,869	96.56%	25
C2	10	Radar	106,826	91.97%	23
C2	10	Infrared	1,541,850	98.12%	29
C3	6	Radar	1,349,766	46.12%	30
C3	6	Infrared	243,798	56.16%	56
C1+C2	31	Radar	120,895	89.51%	6
C1+C2	31	Infrared	195,375	91.52%	5
C1+C2+C3	37	Infrared	165,269	78.29%	7

Table 3: Grid search: Performance metrics of supervised classification models.

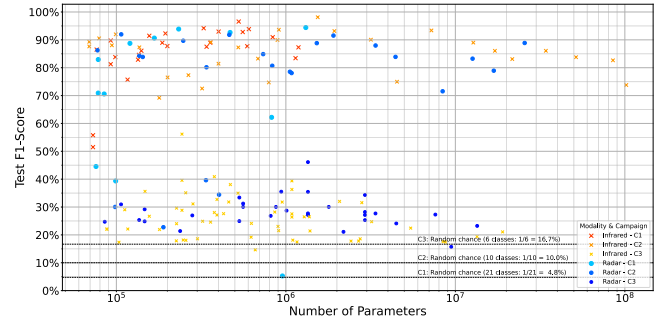


Figure 4: F1-score distribution of our supervised models. Each point represents a unique model configuration evaluated during our hyperparameter search.

models, minimizing training loss, rather than stopping prematurely solely based on validation loss, is critical for test performance, as overly early termination leads to noisy, un-converged models.

Supervised results are summarized in Table 3, with the overall F1-score distribution plotted in Figure 4.

For the challenging emotion classes in campaign 3, we attempted to improve accuracy by using a genetic algorithm to mutate and combine the top-performing architectures. However, this did not yield better models within our computational limits. We expect this likely stems from the subtle, microscopic nature of emotional expressions, which may require denser sensor arrays or more tracking landmarks to capture. Across all campaigns, the radar performed worse than the infrared camera, which we attribute to the radar’s lower spatial resolution, sparser data collection, and higher signal noise.

Our unsupervised models were evaluated using 1-shot classification, where we measure cosine similarity for each query recording with one reference recording per unseen class. This evaluation was repeated using cross-validation across all samples. The results (Table 4) show that the unsupervised models can generalize remarkably well to entirely new behaviors, even matching supervised performance while requiring only a single reference example and significantly fewer parameters.

Campaign	# Behaviors	Modality	# Params	Test F1	# Models
C1	17-2-2	Radar	254,144	93.03%	6
C1	17-2-2	Infrared	142,016	96.68%	7
C2	6-2-2	Radar	93,440	77.51%	14
C2	6-2-2	Infrared	923,280	93.48%	31
C1+C2	23-4-4	Radar	175,776	90.31%	7
C1+C2	23-4-4	Infrared	282,176	88.13%	7
C1+C2+C3	25-6-6	Radar	817,472	79.20%	7

Table 4: Grid search: One-shot classification results for unsupervised models. The behaviors column indicates the split of train, validation, and test classes.

6 Method 3: Neural Network-Based Interpolation and Classification

The radar-based dataset consists of 13 separate radar sensors each recording data at different timestamps, in order to unify the sensors to a single timeline, interpolation is required. This method consists of performing interpolation using Neural Networks and comparing with classical methods, emotion classification is also performed.

6.1 Preprocessing

Using the preprocessing sample code provided in the dataset, each of the 13 radar sensors' data were converted into 3 values corresponding to the XYZ axis, afterwards, the timestamps were rounded to 1 decimal place and the sensors concatenated together into a single csv file resulting in 39 values at each timestamp for each user for each emotion. These were then concatenated together to get a singular csv file containing the processed data of every user, emotion and sensor. Grouping by user_id and emotion, a sparse sequence of observations from all the sensors combined can be retrieved.

To avoid any overlap between dataset splits and to test on unseen users, the user_ids were split into train, valid and test sets with ratios 60%/20%/20%. Normalization is performed by setting mean to 0 to allow the model to only train on the residual. Min-max normalization is applied to the timestamps to set the minimum to 0 and maximum to 1.

While training the models, 10% to 30% of the sequence is randomly set aside as the target sequence, this is set to 20% for validation and testing with a static random state for reproducibility. An observation mask containing the non-Nan values is obtained for both the input and target sequence, this is combined with the input sequence to allow the model to focus on observations and is used to train the model using only non-Nan target values. Last Observation Carried Forward (LOCF) is then applied on the data which fills NaN or empty values with the last previously observed value in the sequence to counter-act the sparsity of the dataset.

6.2 Model Selection

The selected models used for interpolation and classification are Neural Controlled Differential Equation (NCDE) model by Kidger et. al [Kidger et al., 2020], a 1D U-Net model and a Temporal Convolutional Network (TCN). NCDE was selected due to its ability to deal with irregular timestamps and obtain output at any timestamp. As shown in Figure. 5,

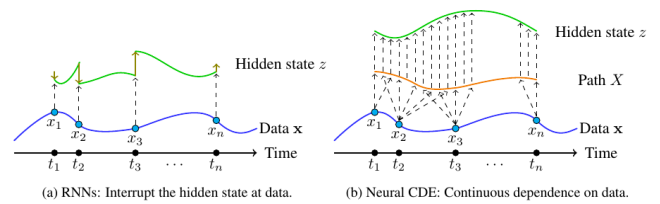


Figure 5: Comparison of hidden state updates between RNN and NCDE taken from the NCDE paper.

the main advantage over Recurrent Neural Networks (RNNs) is the continuous update of its hidden state. This continuous update is achieved by solving and integrating the differential equation in Equation. 2 where $f(t, z(t))$ and $g(x_0)$ are Neural Networks (NNs) and X is a cubic Hermite spline interpolation of the sparse input sequence to make it a differentiable continuous data stream. A cubic Hermite spline is a piecewise function where each piece is a third degree polynomial whose values and first derivatives at the endpoints of the domain interval are the same as the given function or input [?].

$$\frac{dz(t)}{dt} = f(t, z(t)) \frac{dX(t)}{dt}, \quad z(t_0) = g(x_0), \quad (2)$$

A U-Net is a Convolutional Neural Network (CNN) whose input and output are the same shape achieved by continuously downsampling via Max pooling and then upsampled using transpose convolution layers with skip connections resulting in a U shape, the classifier head is attached to the bottleneck located between the downsampling and upsampling layers and consists of an adaptive average pooling to reduce the size to 1 along with a simple MLP classifier. Instead of the predicted values being obtained separately by inputting target timestamps, the input sequence has dummy values at those positions or indices and later retrieved from the output at those same indices. The model is expected to fill in the blanks, so to speak.

A TCN is a CNN with causal padding where successive layers have greater dilation allowing each point of the output to see the entire most of the surrounding input sequence. The classifier head is a linear layer attached to the end of the output sequence. It has the same input as U-Net where the model learns to fill the dummy values at target timestamps.

To compare performance, the baseline is a scikit-learn interpolator with a 3 degree spline transformer with 4 knots followed by a ridge with $\alpha = 0.001$. This will be used to compare the interpolation performance. U-Net and TCN were also trained in an ensemble with the baseline where the loss was calculated using model output and the difference between target values and interpolated values, the mean was not normalized to mean 0 for these experiments.

Shown in Figure. 6 is the workflow of the NCDE model, it starts with interpolating the input sequence using a differentiable hermite cubic spline to obtain a continuous control data X , whose first instance $x(0)$ is passed through an MLP to get $z(0)$ and both are fed into the cdeint function along with the target timestamps dictating at what time we want the hidden states $z(t)$, the output sequence is fed into an MLP decoder to

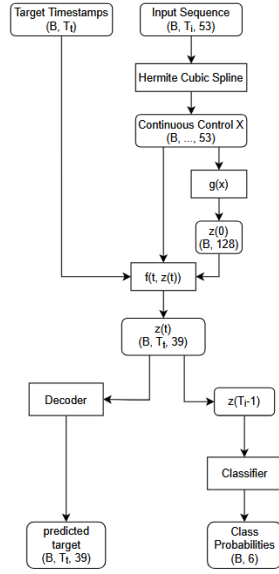


Figure 6: Workflow of the proposed method. It uses a differentiable Hermite cubic spline that can be integrated to obtain the hidden state at arbitrary timestamps. The final hidden state is used to obtain class probabilities, similar to RNNs.

Hyperparameter	Value
Dataset splits	60%/20%/20%
Hidden channels	128
Input channels	53
Output channels	39
Batch size	1
Gradient Accumulation	8
Learning rate	1×10^{-4}
Optimizer	AdamW
Number of epochs	20
Gradient clipping max norm	1.0
Interpolation	Cubic Spline
ODE solver	RK4

Table 5: Hyperparameters used for training the NCDE model.

Method	MSE	Cross-Entropy	Accuracy
Baseline	0.0962	-	-
NCDE model	0.0999	3.6167	44.44
U-Net	0.0995	1.7920	16.66
TCN	0.0992	1.7444	25
U-Net + Baseline	0.0962	1.7920	16.66
TCN + Baseline	0.1039	1.6496	33.33

Table 6: NCDE model performance on test set.

get the predicted values while the final hidden state is fed into an MLP classifier to get emotion probabilities.

6.3 Metrics

The model was trained using a combination of Mean Squared Error (MSE) loss for the sensor interpolation and Cross-Entropy loss for the emotion classification. Accuracy was used to measure the classification performance of the model on the test set, it was used in place of more complex metrics like F1-score due to the balanced nature of the dataset i.e. the number of samples for each emotion class are roughly equal.

MSE is shown in Equation. 3 where N is the number of reconstructed sensor values, y_i is the ground truth value, and $\hat{y}_i$ is the corresponding prediction. Minimizing the MSE encourages the reconstructed values to closely match the original observations.

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad (3)$$

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{p}_{i,c}), \quad (4)$$

Equation. 4 shows the cross-entropy loss where C is the number of classes, $y_{i,c}$ is the one-hot encoded ground-truth label for sample i , and $\hat{p}_{i,c}$ is the predicted probability of class c . The loss penalizes incorrect predictions while encouraging the model to assign high probability to the correct class. In inference, argmax is used to select the class.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i = y_i), \quad (5)$$

Equation. 5 shows how to calculate accuracy where N is the total number of samples.

6.4 Hyperparameters

Table. ?? shows the hyperparameters, batch size was set to 1 as the length of the target sequences varied, to counteract this, gradient accumulation every 8 steps was implemented mimic the effect of having a batch size of 8. Gradient clipping was implemented to avoid exploding gradients,

6.5 Results

The models were trained for 20 epochs and the weights with the best MSE loss and cross-entropy loss on valid set were saved along with final epoch weights. The performance of the models on the test set is listed in Table. 6. None of the models were able to surpass the baseline, the closest was by the U-Net and baseline ensemble, meaning that the Neural Networks were unable to improve upon the cubic spline. The best classification performance was by the NCDE model at 44.44% accuracy.

As both the baseline and Neural Networks have poor MSE loss, the problems lies in the dataset, specifically, the small number of samples (181 samples) and possibly may not have

519 enough discriminatory information. The poor classification
 520 performance can partly be attributed to the model performing
 521 both classification and interpolation, additionally, classifica-
 522 tion was not performed on the entire sequence due to setting
 523 20% aside for interpolation resulting in less information for
 524 the classifier.

525 7 Discussion

526 Feature-based classification using classical machine learning
 527 achieved promising results despite several limitations. The
 528 primary limitation was the small number of training samples:
 529 after preprocessing and windowing, only approximately 600
 530 samples remained, restricting the models' ability to general-
 531 ize. This could be addressed by collecting additional data,
 532 applying feature-level augmentation (e.g., noise injection), or
 533 generating synthetic samples using generative models. Per-
 534 formance could also benefit from incorporating additional
 535 modalities, such as infrared data, allowing the model to ex-
 536 ploit complementary information alongside radar features.

537 The deep learning approach was similarly constrained by
 538 the choice of architecture. Our Transformer encoders were
 539 adapted from general-purpose language models rather than
 540 architectures specifically designed for point clouds or tem-
 541 poral radar data, and we did not investigate robustness to
 542 different sensor layouts or conduct extensive ablation stud-
 543 ies. Generalization could be improved through data aug-
 544 mentation strategies such as randomly dropping active sen-
 545 sors, downsampling dense radar frames, removing temporal
 546 frames, cropping recordings to varying durations, injecting
 547 Gaussian noise into spatial coordinates, and applying geo-
 548 metric transformations including rotations, scaling, and trans-
 549 lations. Furthermore, the proposed framework is compatible
 550 with all raw modalities available in the RF-Behavior dataset.
 551 A unified architecture with a shared temporal backbone and
 552 modality-specific input encoders could enable joint training
 553 across all sensor types and behaviors. Aligning the learned
 554 sensor representations with a pre-trained text encoder could
 555 further enable zero-shot recognition of previously unseen be-
 556 haviors from textual descriptions.

557 The interpolation approach was also limited by the small
 558 dataset size. Incorporating additional modalities, particularly
 559 infrared data, could improve interpolation by exploiting com-
 560plementary temporal patterns. Extending interpolation across
 561 all modalities simultaneously would further increase the like-
 562lihood of reconstructing missing information when individual
 563 sensors are inactive. Classification performance is also ex-
 564 pected to improve with richer feature representations; how-
 565 ever, training the classifier on complete temporal sequences
 566 rather than individual interpolated segments would likely pro-
 567 duce more robust predictions. Future work could also in-
 568 vestigate classification performance on subsets of sentiment
 569 classes to better understand the model's strengths and limita-
 570 tions.

571 References

572 [Ain *et al.*, 2017] Qurat Tul Ain, Mubashir Ali, Amna Riaz,
 573 Amna Noureen, Muhammad Kamran, Babar Hayat, and

A-u Rehman. Sentiment analysis using deep learning tech- 574
 niques: a review. *International Journal of Advanced Com-* 575
puter Science and Applications, 8(6), 2017. 576

[Alsmadi *et al.*, 2009] Mutasem khalil Alsmadi, Khairud- 577
 din Bin Omar, Shahrul Azman Noah, and Ibrahim Al- 578
 marashdah. Performance comparison of multi-layer per- 579
 ceptron (back propagation, delta rule and perceptron) al- 580
 gorithms in neural networks. In *2009 IEEE International* 581
Advance Computing Conference, pages 296–299, 2009. 582

[Chen *et al.*, 2020] Ting Chen, Simon Kornblith, Moham- 583
 mad Norouzi, and Geoffrey Hinton. A simple framework 584
 for contrastive learning of visual representations. In *In-* 585
ternational conference on machine learning, pages 1597– 586
 1607. PmLR, 2020. 587

[Deng *et al.*, 2018] Hai Deng, Zhe Geng, and Braham 588
 Himed. Radar target detection using target features and 589
 artificial intelligence. In *2018 International Conference* 590
on Radar (RADAR), pages 1–4, 2018. 591

[Devlin *et al.*, 2019] Jacob Devlin, Ming-Wei Chang, Ken- 592
 ton Lee, and Kristina Toutanova. Bert: Pre-training of 593
 deep bidirectional transformers for language understand- 594
 ing. In *Proceedings of the 2019 conference of the North* 595
American chapter of the association for computational lin- 596
guistics: human language technologies, volume 1 (long 597
and short papers), pages 4171–4186, 2019. 598

[Froehlich *et al.*, 2024] Ann-Christine Froehlich, Desir 599
 Mejdani, Lukas Engel, Johanna Braeunig, Christoph 600
 Kammel, Martin Vossiek, and Ingrid Ullmann. A 601
 millimeter-wave mimo radar network for human activity 602
 recognition and fall detection. In *2024 IEEE Radar* 603
Conference (RadarConf24), pages 1–5, 2024. 604

[Kazemi *et al.*, 2022] Alireza Kazemi, Reza Boostani, Mah- 605
 moud Odeh, and Mohammad Rasmi AL-Mousa. Two- 606
 layer svm, towards deep statistical learning. In *2022 In-* 607
ternational Engineering Conference on Electrical, Energy, 608
and Artificial Intelligence (EICCEAI), pages 1–6, 2022. 609

[Kidger *et al.*, 2020] Patrick Kidger, James Morrill, James 610
 Foster, and Terry Lyons. Neural Controlled Differential 611
 Equations for Irregular Time Series. *Advances in Neural* 612
Information Processing Systems, 2020. 613

[Manjunath *et al.*, 2018] Ankith Manjunath, Ying Liu, 614
 Bernardo Henriques, and Armin Engstle. Radar based 615
 object detection and tracking for autonomous driving. 616
 In *2018 IEEE MTT-S International Conference on Mi-* 617
crowaves for Intelligent Mobility (ICMIM), pages 1–4, 618
 2018. 619

[Perangin-Angin and Bachtiar, 2021] Dariswan Janweri 620
 Perangin-Angin and Fitra A. Bachtiar. Classification of 621
 stress in office work activities using extreme learning 622
 machine algorithm and one-way anova f-test feature se- 623
 lection. In *2021 4th International Seminar on Research of* 624
Information Technology and Intelligent Systems (ISRITI), 625
 pages 503–508, 2021. 626

[Radford *et al.*, 2021] Alec Radford, Jong Wook Kim, Chris 627
 Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agar- 628
 wal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack 629

630 Clark, et al. Learning transferable visual models from nat-
631 ural language supervision. In *International conference on*
632 *machine learning*, pages 8748–8763. PmLR, 2021.

633 [Rokade *et al.*, 2025] Gayatri Rokade, Radhe Ughade, and
634 Palash Gaurshettiwar. Deep learning for sentiment anal-
635 ysis. In *2025 4th International Conference on Sentiment*
636 *Analysis and Deep Learning (ICSADL)*, pages 240–244,
637 2025.

638 [Ryan and Massaron, 2025] Mark Ryan and Luca Massaron.
639 2025.

640 [Sun *et al.*, 2020] Yuliang Sun, Tai Fei, Xibo Li, Alexan-
641 der Warnecke, Ernst Warsitz, and Nils Pohl. Real-
642 time radar-based gesture detection and recognition built
643 in an edge-computing platform. *IEEE Sensors Journal*,
644 20(18):10706–10716, 2020.

645 [Touvron *et al.*, 2023] Hugo Touvron, Thibaut Lavril, Gau-
646 tier Izacard, Xavier Martinet, Marie-Anne Lachaux, Tim-
647 othée Lacroix, Baptiste Rozière, Naman Goyal, Eric Ham-
648 bro, Faisal Azhar, et al. Llama: Open and efficient founda-
649 tion language models. *arXiv preprint arXiv:2302.13971*,
650 2023.

651 [Vaswani *et al.*, 2017] Ashish Vaswani, Noam Shazeer, Niki
652 Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez,
653 Łukasz Kaiser, and Illia Polosukhin. Attention is all you
654 need. *Advances in neural information processing systems*,
655 30, 2017.

656 [Zuo *et al.*, 2025] Si Zuo, Yuqing Song, Sahar Golipoor,
657 Ying Liu, Xujun Ma, and Stephan Sigg. Rf-behavior: A
658 multimodal radio-frequency dataset for human behavior
659 and emotion analysis, 2025.